

INFORME DE LABORATORIO: ARQUITECTO CLOUD

Asignatura: Sistemas Operativos

Universidad del Valle - Sede Santander de Quilichao Grupo: 05

Integrantes: Johan Mosquera Orejuela, Jhoyner Cordoba, Jhon Diego Savedra, Santiago Tejada

Rojas

Docente: José Alexander Castañeda Muñoz

Fecha: 11/06/2026

1. INTRODUCCIÓN Y OBJETIVOS

El presente informe documenta el diseño, configuración y despliegue de una infraestructura elástica e híbrida micro-cloud. El objetivo principal es aplicar conceptos fundamentales de los sistemas operativos modernos (gestión de memoria, aislamiento de procesos, redes internas, sistemas de archivos compartidos y orquestación elástica) combinando entornos de virtualización de Tipo 2, contenedores Docker y un clúster local de Kubernetes (Minikube).

2. COMPONENTE 1: VIRTUALIZACIÓN BASADA EN TIPO 2

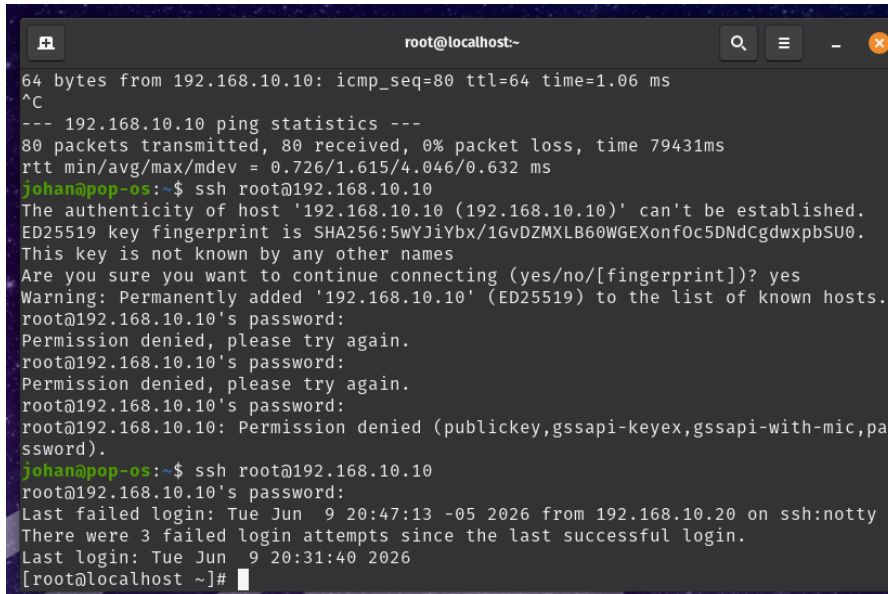
Para el aislamiento de la infraestructura se utilizó el hipervisor Oracle VM VirtualBox en un sistema anfitrión Windows. Se desplegaron dos máquinas virtuales (VM) con arquitecturas de sistemas operativos diferentes para simular un entorno empresarial híbrido:

- **VM1 (Servidor Principal):** Pop!_OS 22.04 LTS (Distribución basada en Debian/Ubuntu).
- **VM2 (Servidor Secundario):** Rocky Linux 9.4 (Distribución basada en Red Hat Enterprise Linux - RHEL).

2.1. Red de Interconexión y Seguridad (SSH)

Se configuró una **Red Interna Privada** en VirtualBox para garantizar que ambas máquinas virtuales se comuniquen de forma aislada sin interferencia del tráfico externo. Las direcciones IP estáticas se asignaron en el rango 192.168.10.X.

Para la administración segura, se habilitó el demonio SSH (sshd) en Rocky Linux y se configuraron las llaves criptográficas en Pop!_OS. Se validó la conexión directa mediante la terminal, asegurando la administración remota sin interfaz gráfica.



```
root@localhost:~#
64 bytes from 192.168.10.10: icmp_seq=80 ttl=64 time=1.06 ms
^C
--- 192.168.10.10 ping statistics ---
80 packets transmitted, 80 received, 0% packet loss, time 79431ms
rtt min/avg/max/mdev = 0.726/1.615/4.046/0.632 ms
johan@pop-os:~$ ssh root@192.168.10.10
The authenticity of host '192.168.10.10 (192.168.10.10)' can't be established.
ED25519 key fingerprint is SHA256:5wYJiYbx/1GvDZMXLB60WGEXonf0c5DNDcGdwpxbSU0.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.10.10' (ED25519) to the list of known hosts.
root@192.168.10.10's password:
Permission denied, please try again.
root@192.168.10.10's password:
Permission denied, please try again.
root@192.168.10.10's password:
root@192.168.10.10: Permission denied (publickey,gssapi-keyex,gssapi-with-mic,password).
johan@pop-os:~$ ssh root@192.168.10.10
root@192.168.10.10's password:
Last failed login: Tue Jun  9 20:47:13 -05 2026 from 192.168.10.20 on ssh:notty
There were 3 failed login attempts since the last successful login.
Last login: Tue Jun  9 20:31:40 2026
[root@localhost ~]#
```

3. COMPONENTE 2: CONTENEDORIZACIÓN ESTANDARIZADA

En la VM1 (Pop!_OS) se instaló y configuró **Docker Engine**. A diferencia de la virtualización clásica, Docker proporciona aislamiento a nivel de espacio de nombres (*namespaces*) y grupos de control (*cgroups*), compartiendo el mismo núcleo (*kernel*) del sistema operativo anfitrión.

Se diseñó una arquitectura multi-contenedor para empaquetar servicios web. Se creó una imagen personalizada basada en Nginx y se mapearon los volúmenes del sistema de archivos local hacia el directorio interno del contenedor (/usr/share/nginx/html). Esto permite la persistencia de datos y la actualización del contenido de la documentación técnica en tiempo real sin necesidad de reiniciar el contenedor.

4. COMPONENTE 3: ORQUESTACIÓN E INFRAESTRUCTURA ELÁSTICA (KUBERNETES)

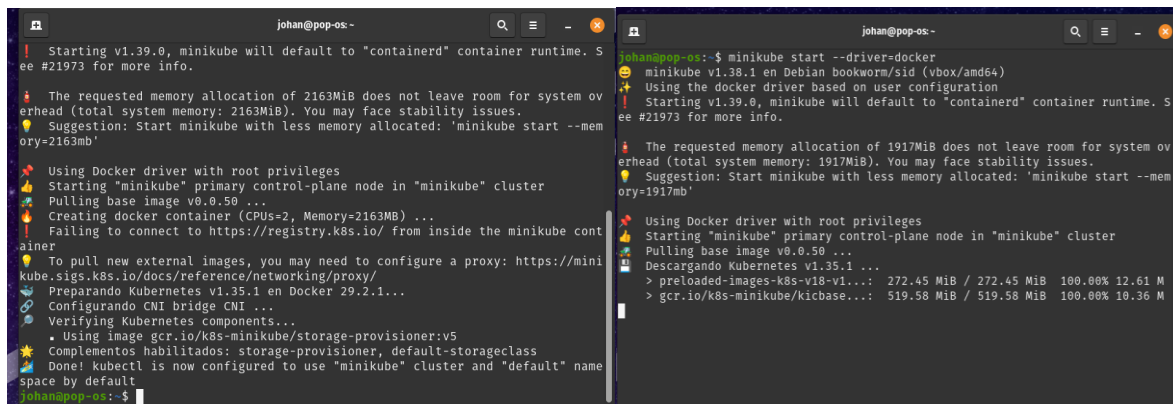
Para gestionar el ciclo de vida de los contenedores a gran escala, se desplegó un clúster de **Minikube** utilizando el driver de Docker en Pop!_OS.

4.1. Resolución del Problema de Asignación de Recursos (RAM)

Durante el despliegue inicial, Minikube falló debido al error crítico `RSRC_INSUFFICIENT_REQ_MEMORY`, ya que la máquina virtual base tenía menos de los 1800MB mínimos requeridos para el plano de control (*control-plane*). La solución aplicada consistió en apagar la VM, ingresar a los ajustes de VirtualBox y expandir la memoria RAM de la máquina virtual Pop!_OS a **4096 MB (4GB)** y asignar **2 CPUs**. Tras esto, el entorno arrancó correctamente mediante el comando:

Bash

```
minikube start --driver=docker --memory=2200mb
```



The image shows two terminal windows side-by-side. The left window shows the initial attempt to start minikube with the default 2163MB memory, which fails with an error: 'The requested memory allocation of 2163MiB does not leave room for system overhead (total system memory: 2163MiB). You may face stability issues. Suggestion: Start minikube with less memory allocated: 'minikube start --memory=2163mb''. The right window shows the successful execution of the command 'minikube start --driver=docker --memory=2200mb'. It shows the minikube v1.38.1 en Debian bookworm/sid (vbox/amd64) using the docker driver based on user configuration. It starts v1.39.0, minikube will default to "containerd" container runtime. Suggestion: Start minikube with less memory allocated: 'minikube start --memory=1917mb'. It then shows the successful execution of the command 'minikube start --driver=docker --memory=2200mb' and the successful execution of the command 'minikube start --driver=docker --memory=2200mb'.

4.2. Despliegue Declarativo (deployment.yaml)

Se creó un manifiesto declarativo en formato YAML para definir un *Deployment* y un *Service*. El Deployment asegura la alta disponibilidad manteniendo un estado de **2 réplicas (Pods)** del servidor web Nginx de forma simultánea.

YAML

apiVersion: apps/v1

kind: Deployment

metadata:

name: nginx-deployment

spec:

replicas: 2

selector:

matchLabels:

app: nginx

template:

metadata:

labels:

app: nginx

spec:

containers:

- name: nginx

image: nginx:latest

ports:

- containerPort: 80

apiVersion: v1

kind: Service

metadata:

name: nginx-service

spec:

type: NodePort

selector:

app: nginx

ports:

- port: 80

targetPort: 80

nodePort: 30080

Se aplicó el archivo al clúster ejecutando:

Bash

kubectl apply -f deployment.yaml

4.3. Verificación del Estado de los Componentes (Evidencias Obligatorias)

Para comprobar el correcto funcionamiento exigido en la guía, se ejecutaron las herramientas de diagnóstico de kubectl:

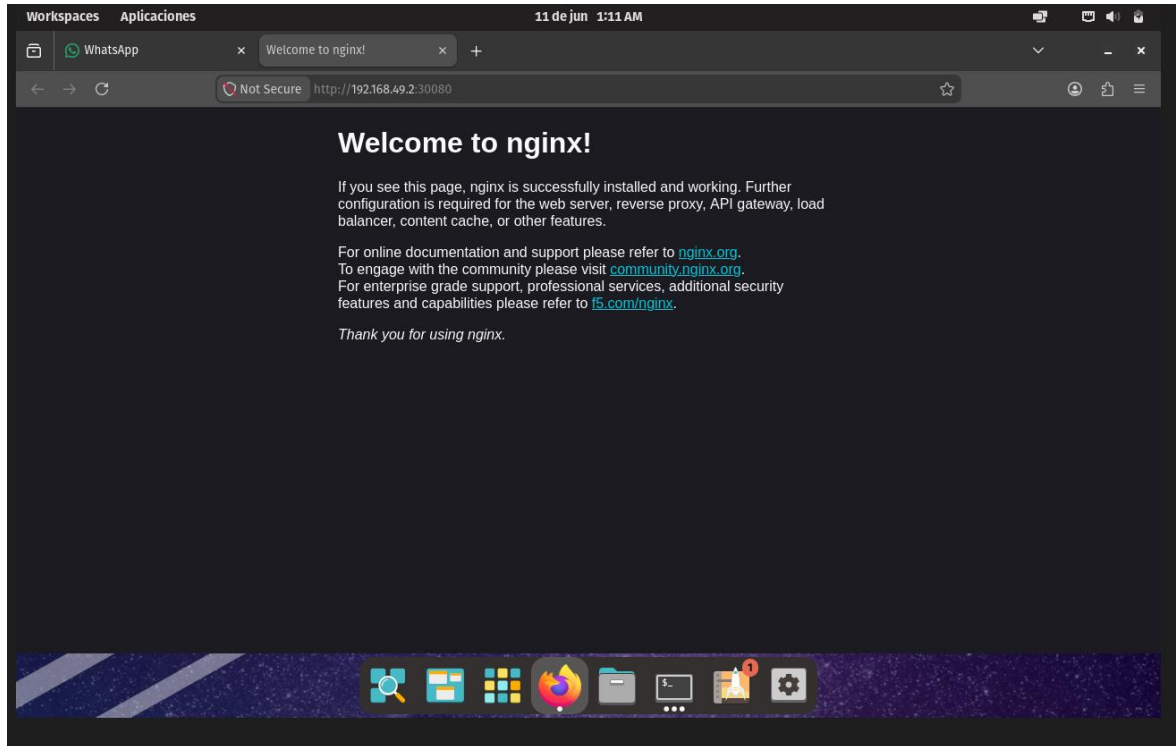
1. **Estado de los Pods (kubectl get pods):** Se verificó que las dos réplicas programadas se crearon y pasaron exitosamente al estado Running.
2. **Estado del Servicio (kubectl get svc):** Se confirmó la creación de nginx-service bajo la modalidad de **NodePort**, exponiendo el puerto interno 80 a través del puerto asignado del nodo virtual (30080).

```
johan@pop-os: ~/proyecto_k8s
johan@pop-os:~/proyecto_k8s$ cd ~/proyecto_k8s
minikube update-context
👉 The control-plane node minikube host does not exist
👉 To start a cluster, run: "minikube start"
johan@pop-os:~/proyecto_k8s$ minikube update-context
😬 No changes required for the "minikube" context
💖 Contexto actual "minikube"
johan@pop-os:~/proyecto_k8s$ cd ~/proyecto_k8s
johan@pop-os:~/proyecto_k8s$ minikube update-context
😬 No changes required for the "minikube" context
💖 Contexto actual "minikube"
johan@pop-os:~/proyecto_k8s$ kubectl apply -f deployment.yaml
deployment.apps/nginx-deployment created
service/nginx-service created
johan@pop-os:~/proyecto_k8s$ kubectl get pods
NAME                                READY   STATUS              RESTARTS   AGE
nginx-deployment-59f86b59ff-zhbqq    0/1     ContainerCreating   0           37s
nginx-deployment-59f86b59ff-zkw7n    0/1     ContainerCreating   0           37s
johan@pop-os:~/proyecto_k8s$ kubectl get svc
NAME             TYPE       CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes       ClusterIP  10.96.0.1    <none>        443/TCP          3m6s
nginx-service    NodePort   10.109.164.246 <none>        80:30080/TCP    67s
johan@pop-os:~/proyecto_k8s$
```

4.4. Acceso desde el Navegador Web Anfitrión

Para validar el mapeo de red elástica, se consultó la dirección IP asignada al clúster de Minikube (192.168.49.2). Al ingresar dicha IP desde el navegador web Firefox de la máquina, el servicio enrutó la petición hacia los Pods internos, mostrando la pantalla oficial de bienvenida de

Nginx.



5. COMPONENTE 4: SITIO WEB DE DOCUMENTACIÓN Y PORTAL PÚBLICO

Como parte del entregable de documentación del proyecto, se construyó una plataforma web responsiva utilizando HTML5 y CSS3 estructurada en las secciones obligatorias: *Home*, *Equipo*, *Componentes Implementados* y *Conclusiones*.

5.1. Control de Versiones y Publicación en GitHub

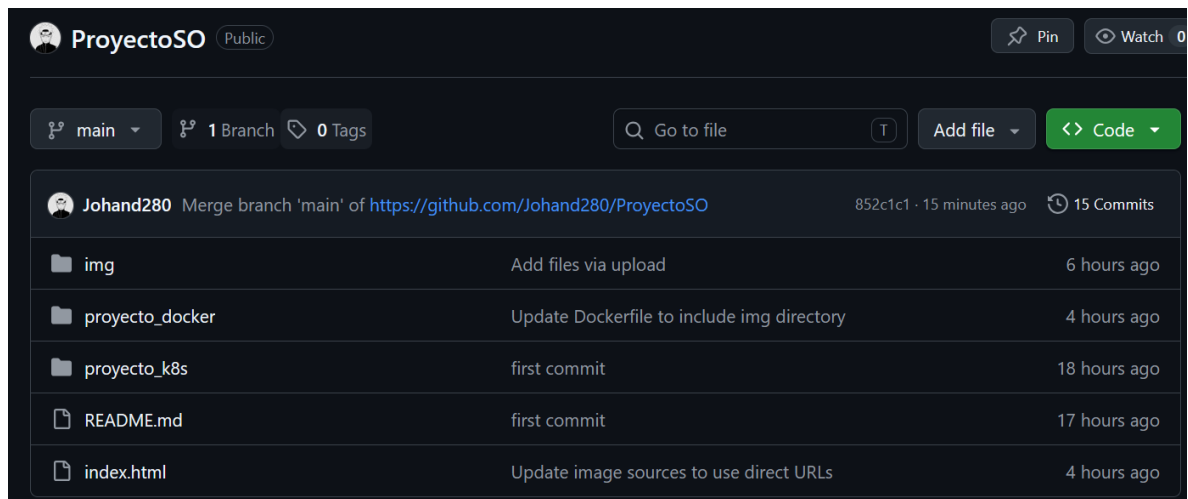
El proyecto se gestionó localmente mediante Git. Tras resolver un conflicto de identidad inicial en la máquina virtual, se configuraron las variables globales de usuario y correo:

Bash

```
git config --global user.email "tu_correo@example.com"
```

```
git config --global user.name "Tu Nombre"
```


Los archivos fueron cargados en el repositorio público de GitHub bajo la cuenta del integrante **Johan** (Johand280/ProyectoSO). Posteriormente, se activó la herramienta **GitHub Pages** en la rama principal (main), lo que generó un enlace público en internet para que el sitio web de documentación técnica sea completamente accesible para el cuerpo docente.



6. CONCLUSIONES

1. **Estandarización del Entorno:** La adopción de contenedores Docker demostró solucionar el problema de inconsistencia de entornos ("funciona en mi máquina"), permitiendo que la aplicación corra de forma idéntica en Pop!_OS y Rocky Linux.
2. **Alta Disponibilidad Garantizada:** El uso de Kubernetes mediante políticas declarativas asegura la resiliencia de la infraestructura; si uno de los Pods falla o es eliminado, el clúster regenera automáticamente una nueva réplica para mantener el servicio activo.
3. **Optimización de Recursos Físicos:** El laboratorio permitió comprender la diferencia crítica entre la sobrecarga de RAM que genera la virtualización tradicional y la ligereza y eficiencia en el consumo de memoria que ofrece la orquestación de contenedores en la nube.